JavaScript Mastery: Zero to Hero

A complete study report: concepts, pros and cons, and interview questions with answers. Prepared October 2026.

How to use this report

Read one part, then test yourself in chat. I will quiz you one question at a time and only move on when you have shown you understand each concept. Type the code yourself in the browser console or Node; do not just read it.

Roadmap

1. Pros and cons of JavaScript
2. Core language: values, types, operators, control flow
3. Functions, scope, hoisting, closures, `this`
4. Objects, arrays, prototypes, classes
5. Modern ES6+ features
6. Asynchronous JavaScript
7. The browser and the DOM
8. Errors, modules, performance, security
9. Interview questions with answers
10. A 12-week mastery plan

---

1. Pros and cons of JavaScript

Pros

- **Runs everywhere.** Every browser ships a JavaScript engine. With Node.js, Deno and Bun it runs on servers, and with React Native or Electron on phones and desktops.
- **One language for the full stack.** Frontend and backend can share language, validation logic and types.
- **Huge ecosystem.** npm is the largest package registry in the world. React, Vue, Svelte, Angular, Next.js and Express are mature.
- **Beginner friendly.** No compile step; open the browser console and start.

- **Flexible paradigms.** Procedural, object-oriented (prototype-based) and functional styles all work.
- **First-class functions.** Functions are values, enabling callbacks, higher-order functions and closures.
- **Strong async model.** The event loop, promises and async/await suit I/O-heavy work.
- **Fast engines.** V8, SpiderMonkey and JavaScriptCore use JIT compilation.
- **Huge community and job market.**
- **Evolving yearly.** ECMAScript releases keep adding useful features.

Cons

- **Surprising type coercion.** `[] + {}`, `'5' - 2`, `0 == ''`. Use `===`.
- **Dynamic typing.** Type errors show up at runtime. TypeScript is the common fix.
- **Single-threaded main thread.** Heavy CPU work blocks the UI unless moved to Web Workers or worker threads.
- **Legacy quirks.** `var`, `typeof null === 'object'`, `0.1 + 0.2 !== 0.3` (floating point, shared by most languages).
- **Fragmented tooling.** Bundlers, transpilers and frameworks change quickly; "JavaScript fatigue".
- **Security exposure.** XSS, prototype pollution, npm supply-chain attacks.
- **Weak for heavy computation** compared with C++, Rust or Go.
- **Easy to write messy code.** Freedom without discipline becomes unmaintainable.

One-line interview answer: JavaScript is the only language native to the browser, with a vast ecosystem and strong async support, but its dynamic typing and legacy quirks demand discipline, linting and often TypeScript.

---

2. Core language

2.1 Running JavaScript

- Browser: DevTools console (F12), or a `<script>` tag.
- Node.js: `node file.js`.
- Output: `console.log(value)`.

2.2 Variables

```
var a = 1;   // function-scoped, hoisted as undefined, can redeclare. Avoid.
let b = 2;   // block-scoped, reassignable, temporal dead zone
const c = 3; // block-scoped, cannot be reassigned (object contents can still
change)
```

Rule: default to `const`, use `let` when you must reassign, never `var`.

2.3 Data types

Seven primitives plus objects:

- `string`, `number`, `bigint`, `boolean`, `undefined`, `null`, `symbol`
- `object` (includes arrays, functions, dates, maps, sets)

Primitives are immutable and copied by value. Objects are copied by reference.

```
typeof 42;          // 'number'
typeof 'hi';        // 'string'
typeof undefined;   // 'undefined'
typeof null;        // 'object'  (historic bug)
typeof [];          // 'object'
typeof (() => {});  // 'function'
Array.isArray([]);  // true
```

Number gotchas: `NaN` is a number and `NaN !== NaN` (use `Number.isNaN`). Safe integers go up to `2**53 - 1`; use `BigInt` (`123n`) beyond that.

2.4 Type coercion

JavaScript converts types implicitly.

```
'5' + 2;   // '52'  (+ prefers string concatenation)
'5' - 2;   // 3     (- forces numbers)
true + 1;  // 2
[] + [];   // ''

// Falsy values (everything else is truthy):
// false, 0, -0, 0n, '', null, undefined, NaN
```

`==` allows coercion; `===` does not. Prefer `===`. `Object.is(a, b)` differs from `===` only for `NaN` and `+0/-0`.

2.5 Operators

- Arithmetic: `+ - * / % **`
- Comparison: `=== !== < > <= >=`
- Logical: `&& || !`, with short-circuiting returning the deciding value
- Nullish coalescing: `a ?? b` (b only if a is `null` or `undefined`; unlike `||`, keeps `0` and `''`)
- Optional chaining: `user?.address?.city`
- Logical assignment: `a ||= b`, `a &&= b`, `a ??= b`
- Spread/rest: `...`

2.6 Control flow

```
if (x > 0) { } else if (x === 0) { } else { }

switch (day) { case 1: /* ... */ break; default: /* ... */ }

for (let i = 0; i < 5; i++) { }
for (const item of array) { }      // values of iterables
for (const key in object) { }      // enumerable keys (incl. inherited)
while (cond) { }
do { } while (cond);

cond ? a : b;                       // ternary
```

`break` exits a loop; `continue` skips an iteration. `for...of` works on arrays, strings, maps, sets; `for...in` is for object keys, not arrays.

2.7 Strings

```
const s = `Hello ${name}`;   // template literal
s.length; s.toUpperCase(); s.trim(); s.includes('lo');
s.split(' '); s.slice(1, 3); s.replace('a', 'b'); s.replaceAll('a', 'b');
s.startsWith('He'); s.padStart(5, '0'); s.at(-1);
```

Strings are immutable.

---

3. Functions, scope, closures, `this`

3.1 Ways to define functions

```
function add(a, b) { return a + b; }          // declaration (hoisted fully)
const add2 = function (a, b) { return a + b; }; // expression (not hoisted
usable)
const add3 = (a, b) => a + b;                 // arrow function
```

Defaults, rest and destructuring:

```
function greet(name = 'friend', ...others) { }
function show({ id, title }) { }
```

3.2 Arrow functions vs regular functions

- Arrows have no own `this`, `arguments`, `super` or `new.target`; they capture `this` lexically.
- Arrows cannot be used with `new`.
- Use regular functions for object methods and constructors; arrows for callbacks.

3.3 Scope

- **Global scope**, **function scope** (`var`), **block scope** (`let`, `const`).
- **Lexical scope:** a function sees variables from where it was written, not where it is called.
- The **scope chain** is searched inner to outer until the variable is found or a `ReferenceError` is thrown.

3.4 Hoisting

Declarations are processed before code runs.

```
console.log(a); // undefined  (var hoisted, initialised undefined)
var a = 5;

console.log(b); // ReferenceError (let/const in temporal dead zone)
let b = 5;

hello();        // works: function declarations hoist with their body
function hello() {}
```

3.5 Closures

A closure is a function bundled with the variables of the scope where it was created, kept alive even after that scope has finished.

```
function counter() {
  let count = 0;
  return () => ++count;
}
const c = counter();
c(); // 1
c(); // 2
```

Uses: data privacy, factories, memoization, currying, callbacks, module pattern. Risk: unintended memory retention.

Classic trap:

```
for (var i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 3 3 3
for (let i = 0; i < 3; i++) setTimeout(() => console.log(i)); // 0 1 2
```

`var` shares one `i`; `let` creates a fresh binding per iteration.

3.6 The `this` keyword

`this` is decided by how a function is called:

1. `new Foo()` : the new object
2. `fn.call(obj)`, `apply`, `bind` : `obj` (explicit)
3. `obj.method()` : `obj` (implicit)
4. plain call `fn()` : `undefined` in strict mode, global object otherwise
5. arrow function : `this` of the enclosing scope

```
const user = { name: 'Asha', hi() { return this.name; } };
const f = user.hi;
f();                // undefined (lost this)
f.call(user);       // 'Asha'
const g = user.hi.bind(user);
g();                // 'Asha'
```

3.7 Higher-order functions and functional patterns

A higher-order function takes or returns a function. Know these: `map`, `filter`, `reduce`, `forEach`, `some`, `every`, `find`.

- **Currying:** `add(1)(2)(3)`
- **Composition:** `compose(f, g)(x) = f(g(x))`
- **Pure function:** same input gives same output, no side effects
- **Immediately Invoked Function Expression (IIFE):** `(function () { })();`
- **Memoization:** cache results by arguments

---

4. Objects, arrays, prototypes, classes

4.1 Objects

```
const person = { name: 'Ravi', age: 30, greet() { return `Hi ${this.name}`; }
};
person.name; person['age'];
person.city = 'Kolkata'; delete person.city;
'name' in person; person.hasOwnProperty('name'); Object.hasOwn(person,
'name');
Object.keys(person); Object.values(person); Object.entries(person);
Object.assign({}, person); const copy = { ...person };
Object.freeze(person);   // shallow immutability
```

Shallow vs deep copy: spread and `Object.assign` copy only one level. Use `structuredClone(obj)` for a deep copy (not for functions). `JSON.parse(JSON.stringify(x))` drops functions, `undefined`, and dates become strings.

Destructuring:

```
const { name, age: years = 18, ...rest } = person;
const [first, , third] = [1, 2, 3];
```

4.2 Arrays

```
const arr = [1, 2, 3];
arr.push(4); arr.pop(); arr.shift(); arr.unshift(0);   // mutate
arr.slice(1, 3); arr.splice(1, 1);                      // slice copies,
splice mutates
arr.map(x => x * 2); arr.filter(x => x > 1);
arr.reduce((sum, x) => sum + x, 0);
arr.find(x => x > 1); arr.findIndex(x => x > 1); arr.includes(2);
arr.flat(); arr.flatMap(f); arr.sort((a, b) => a - b);
[...arr].reverse(); arr.toSorted(); arr.toReversed(); arr.at(-1);
```

`sort()` without a comparator sorts as strings. Newer non-mutating methods (`toSorted` , `toReversed` , `with`) return copies.

4.3 Map, Set, WeakMap, WeakSet

- `Map` : key-value with any key type, keeps insertion order.
- `Set` : unique values.
- `WeakMap` / `WeakSet` : keys are held weakly (garbage-collectable), not iterable; used for private metadata.

4.4 Prototypes and inheritance

Every object has an internal link to another object, its prototype. Property lookup walks this **prototype chain** until `null` .

```
function Animal(name) { this.name = name; }
Animal.prototype.speak = function () { return this.name + ' makes a sound';
};
const a = new Animal('Dog');
a.__proto__ === Animal.prototype; // true
Object.getPrototypeOf(a) === Animal.prototype; // true
```

`new` does four things: creates an empty object, links its prototype to `Constructor.prototype` , runs the constructor with `this` set to it, and returns it (unless the constructor returns another object).

4.5 Classes (syntax over prototypes)

```
class Animal {
  #secret = 42;                 // private field
  static count = 0;
  constructor(name) { this.name = name; Animal.count++; }
  speak() { return `${this.name} makes a sound`; }
  get id() { return this.#secret; }
}
class Dog extends Animal {
  constructor(name) { super(name); }
  speak() { return `${super.speak()} - woof`; }
}
```

Classes are not hoisted like functions (they are in the temporal dead zone) and must be called with `new`.

4.6 Other must-knows

- **Symbols**, **iterators and generators** (`function*`, `yield`), **Proxy and Reflect**, **getters and setters**, **tagged templates**.
- **JSON:** `JSON.stringify`, `JSON.parse`.
- **Date:** `new Date()`; use `Intl.DateTimeFormat` for formatting; the newer Temporal API is replacing awkward `Date` behaviour.
- **RegExp:** `/ab+c/gi`, `test`, `match`, `matchAll`, named groups.

---

5. Modern ES6+ features (checklist)

- `let` / `const`, template literals, arrow functions, default/rest/spread, destructuring
- Classes, modules (`import` / `export`), Promises, `async` / `await`
- `Map` / `Set` / `Symbol`, iterators, generators, `for...of`
- Optional chaining `?.`, nullish coalescing `??`, logical assignment
- `Array.prototype.at`, `flat`, `flatMap`, `Object.fromEntries`, `Object.hasOwn`, `structuredClone`, `toSorted`
- Private class fields `#x`, static blocks, top-level `await`
- `Promise.all`, `allSettled`, `race`, `any`, `Promise.withResolvers`
- `WeakRef`, `BigInt`, dynamic `import()`

---

6. Asynchronous JavaScript

6.1 The engine model

JavaScript runs one thing at a time on a **call stack**. Slow work (timers, network, file I/O) is handed to the environment (browser or Node). When it finishes, its callback is queued and the **event loop** moves it to the stack only when the stack is empty.

- **Macrotask queue:** `setTimeout`, `setInterval`, I/O, UI events.
- **Microtask queue:** promise callbacks (`.then`, `await` continuations), `queueMicrotask`, `MutationObserver`.
- After each macrotask, **all microtasks run** before the next macrotask. Microtasks therefore beat timers.

```
console.log('1');
setTimeout(() => console.log('2'), 0);
Promise.resolve().then(() => console.log('3'));
console.log('4');
// 1 4 3 2
```

6.2 Callbacks to promises to async/await

- **Callbacks** lead to "callback hell" and inverted control.
- **Promise** has three states: pending, fulfilled, rejected. It settles once and is immutable afterwards.

```
fetch(url)
  .then(r => r.json())
  .then(data => console.log(data))
  .catch(err => console.error(err))
  .finally(() => console.log('done'));

async function load() {
  try {
    const r = await fetch(url);
    if (!r.ok) throw new Error(r.status);   // fetch does not reject on HTTP
errors
    return await r.json();
  } catch (e) { console.error(e); }
}
```

- `await` pauses only the async function, not the whole program.
- Sequential: `await a(); await b();` Parallel: `await Promise.all([a(), b()])`.
- `Promise.all` rejects on the first failure; `allSettled` waits for all; `race` takes the first to settle; `any` takes the first to fulfil.
- A forgotten `await` in a loop with `forEach` does not wait; use `for...of` or `Promise.all(map(...))`.

---

7. The browser and the DOM

- **DOM:** a tree representation of the page. Select with `querySelector`, `querySelectorAll`, `getElementById`.
- Modify: `textContent` (safe), `innerHTML` (XSS risk), `classList.add/remove/toggle`, `setAttribute`, `createElement`, `append`, `remove`.
- **Events:** `addEventListener('click', handler, options)`. Phases: capture (down), target, bubble (up). `stopPropagation()` stops travel; `preventDefault()` stops default action.
- **Event delegation:** attach one listener to a parent and check `event.target`; efficient for dynamic lists.
- **Storage:** `localStorage` (persistent), `sessionStorage` (per tab), cookies (sent to server), `IndexedDB` (large data).
- **Network:** `fetch`, `AbortController`, WebSockets, Server-Sent Events.
- **Performance:** debounce (wait until calls stop) vs throttle (at most once per interval); `requestAnimationFrame`; avoid layout thrashing; lazy loading.
- **Workers:** Web Workers for CPU work; Service Workers for offline and caching.

---

8. Errors, modules, performance, security

Errors

```
try { risky(); } catch (e) { console.error(e.message); } finally { cleanup(); }
throw new Error('Something broke');
class AppError extends Error { constructor(m, code) { super(m); this.code = code; } }
```

Common types: `ReferenceError`, `TypeError`, `SyntaxError`, `RangeError`. Promise rejections need `.catch` or `try/await/catch`.

Modules

- ES modules: `export const x`, `export default`, `import { x } from './a.js'`. Static, strict mode, singleton per URL.
- CommonJS (Node legacy): `require` and `module.exports`; synchronous.

Memory and performance

- Garbage collection is mark-and-sweep (reachability).
- Leaks: forgotten timers, detached DOM nodes, accidental globals, listeners never removed, closures holding large data.
- Tools: DevTools Performance and Memory tabs, Lighthouse.

Security

- Prevent **XSS**: never insert untrusted text with `innerHTML`; use CSP and sanitisation.
- **CSRF:** SameSite cookies and tokens.
- Never trust client-side validation alone; never store secrets in frontend code.
- Audit dependencies (`npm audit`), lock versions, avoid `eval`.
- Prototype pollution: reject `__proto__`, `constructor` keys when merging user objects.

---

9. Interview questions with answers

9.1 Beginner

1. What is JavaScript? A high-level, dynamically typed, interpreted/JIT-compiled, multi-paradigm language standardised as ECMAScript, used for web, server and more.

2. Difference between `var`, `let`, `const`? `var` is function-scoped and hoisted as `undefined`; `let`/`const` are block-scoped with a temporal dead zone; `const` cannot be reassigned.

3. `==` vs `===`? `==` coerces types before comparing; `===` compares type and value.

4. `null` vs `undefined`? `undefined` means a value was never assigned; `null` is an intentional empty value. `null == undefined` is true, `null === undefined` is false.

5. What are primitive types? string, number, bigint, boolean, undefined, null, symbol.

6. What is `NaN`? How to check it? "Not a Number", a numeric value from failed math. Use `Number.isNaN(x)`.

7. What is hoisting? Declarations are registered before execution; `var` and function declarations are usable early, `let` / `const` / `class` are not (TDZ).

8. What are template literals? Backtick strings with `${}` interpolation and multi-line support.

9. What is an arrow function? A concise function syntax without its own `this`, `arguments` or `new` capability.

10. What is `typeof null`? `'object'`, a historic bug kept for compatibility.

11. Mutating vs non-mutating array methods? Mutating: `push`, `pop`, `shift`, `unshift`, `splice`, `sort`, `reverse`. Non-mutating: `map`, `filter`, `slice`, `concat`, `toSorted`.

12. `map` vs `forEach`? `map` returns a new array; `forEach` returns `undefined` and is for side effects.

13. What is the spread operator? `...` expands iterables/objects; the rest parameter collects the remainder.

14. What is JSON? A text data format; `JSON.stringify` and `JSON.parse` convert to and from JavaScript values.

15. What is strict mode? `'use strict'` turns silent errors into thrown errors, forbids accidental globals, and makes plain-call `this` `undefined`. Modules and classes are strict automatically.

9.2 Intermediate

16. Explain closures. A function remembers the variables from its defining scope even after that scope exits. Used for private state, factories, memoization.

17. Explain `this`. Determined by call site: `new`, explicit (`call/apply/bind`), implicit (object method), default (global/undefined), arrows are lexical.

18. `call` vs `apply` vs `bind`? `call(thisArg, a, b)` invokes now; `apply(thisArg, [a, b])` invokes now with an array; `bind(thisArg)` returns a new function with `this` fixed.

19. What is the prototype chain? Objects delegate failed property lookups to their prototype, then its prototype, until `null`.

20. Classical vs prototypal inheritance? JavaScript objects inherit directly from other objects; `class` is syntactic sugar over prototypes.

21. What is the event loop? The mechanism that moves queued callbacks onto the empty call stack; microtasks run before the next macrotask.

22. What is a promise? States? An object representing an eventual result: pending, fulfilled, rejected.

23. `Promise.all` vs `allSettled` vs `race` vs `any`? All-or-fail; wait-for-all with statuses; first settled; first fulfilled.

24. What is event bubbling and capturing? Events travel down (capture) to the target then up (bubble). Delegation uses bubbling.

25. Shallow vs deep copy? Shallow copies top-level only; nested references are shared. Use `structuredClone` for deep.

26. What are higher-order functions? Functions that take or return functions: `map`, `filter`, `reduce`.

27. What is currying? Transforming `f(a, b, c)` into `f(a)(b)(c)`.

28. Debounce vs throttle? Debounce fires after activity stops; throttle fires at most once per interval.

29. What is a pure function? No side effects and deterministic output.

30. What are `Map` and `Set` for? Why not plain objects? Maps allow any key type and keep order with O(1) operations; Sets store unique values.

31. `localStorage` vs `sessionStorage` vs cookies? Persistent client-only; per-tab client-only; small, sent with every request, can be `HttpOnly`.

32. What is the temporal dead zone? The period from block start until a `let`/`const` is initialised, during which access throws `ReferenceError`.

33. What is `async/await`? Syntax over promises making asynchronous code read sequentially; an `async` function always returns a promise.

34. What is destructuring? Unpacking values from arrays/objects into variables, with defaults and renaming.

35. What is optional chaining and nullish coalescing? `?.` short-circuits on `null`/`undefined`; `??` supplies a fallback only for `null`/`undefined`.

9.3 Advanced

36. How does `new` work? Create object, set prototype, run constructor with `this`, return object.

37. How does garbage collection work? Reachability-based mark-and-sweep; unreachable objects are freed.

38. What causes memory leaks? Lingering references: timers, listeners, globals, detached DOM nodes, oversized closures.

39. What is a generator? A function that can pause with `yield` and resume; returns an iterator.

40. What are iterators and iterables? An iterable implements `[Symbol.iterator]()` returning an object with `next()` giving `{ value, done }`.

41. What is a Proxy? An object that intercepts operations (get, set, has, delete) on a target; basis of reactive frameworks.

42. What is a Symbol? A unique, immutable primitive used for hidden keys and well-known protocols.

43. Microtask vs macrotask? Microtasks (promises) drain completely after each task and before rendering; macrotasks (timers, I/O) run one per loop turn.

44. Explain CORS. Browsers block cross-origin responses unless the server sends allowing headers; preflight `OPTIONS` requests check non-simple requests.

45. What is a module bundler and tree shaking? Bundlers combine modules; tree shaking removes unused exports from static ES module graphs.

46. What are Web Workers? Background threads with no DOM access, communicating by `postMessage`.

47. What is memoization? Caching function results by arguments.

48. What is the difference between `Object.freeze` and `const`? `const` prevents rebinding; `freeze` prevents property changes (shallowly).

49. What is event delegation and why use it? One parent listener handles many children; saves memory and works for elements added later.

50. How do you cancel a fetch? `AbortController`: pass `signal` and call `abort()`.

51. What is the difference between `Object.create(null)` and `{}`? The first has no prototype, so no inherited keys; useful as a clean dictionary.

52. What is a WeakMap used for? Associating data with objects without preventing their garbage collection.

53. What is TypeScript and why use it? A typed superset compiled to JavaScript; catches type errors before runtime and improves tooling.

54. How would you prevent XSS? Escape output, use `textContent`, sanitise HTML, set Content-Security-Policy, use `HttpOnly` cookies.

55. What is the difference between `defer` and `async` on script tags? Both download in parallel; `defer` runs after parsing in order; `async` runs as soon as downloaded, unordered.

9.4 Output-prediction questions (try before reading answers)

```
// Q1
console.log(typeof NaN);            // 'number'
// Q2
console.log(0.1 + 0.2 === 0.3);     // false
// Q3
console.log([] == false);           // true
// Q4
console.log([1, 2, 3] + [4]);       // '1,2,34'
// Q5
console.log(Math.max());            // -Infinity
// Q6
console.log(parseInt('08'), 1 < 2 < 3, 3 > 2 > 1); // 8 true false
// Q7
var x = 1; { var x = 2; } console.log(x); // 2
let y = 1; { let y = 2; } console.log(y); // 1
// Q8
const o = { a: 1 }; const p = o; p.a = 9; console.log(o.a); // 9
// Q9
console.log(+'', +true, +[], +{}); // 0 1 0 NaN
// Q10
setTimeout(() => console.log('t'), 0);
Promise.resolve().then(() => console.log('p'));
(async () => { console.log('a'); await null; console.log('b'); })();
console.log('s');
// a s p b t
```

9.5 Coding challenges (implement these from memory)

1. Reverse a string; check a palindrome.
2. Find duplicates in an array; remove duplicates.
3. Flatten a nested array without `flat`.
4. Implement `debounce`, `throttle`, `memoize`, `curry`.
5. Implement `Array.prototype.map`, `filter`, `reduce` (polyfills).
6. Implement `bind`, `call`, `apply` polyfills.
7. Deep clone an object; deep equality.
8. Implement `Promise.all`; a simple `Promise` class.
9. Write an event emitter (`on`, `off`, `emit`).
10. FizzBuzz; two-sum; anagram check; first non-repeating character.
11. Implement a `sleep(ms)` function and retry-with-backoff for `fetch`.
12. Implement LRU cache using `Map`.

Example: debounce

```
function debounce(fn, delay) {
  let timer;
  return function (...args) {
    clearTimeout(timer);
    timer = setTimeout(() => fn.apply(this, args), delay);
  };
}
```

---

10. A 12-week mastery plan

Weeks	Focus	Build
1-2	Syntax, types, control flow, functions	Calculator, to-do in console
3-4	Scope, closures, `this`, objects, arrays	Array method polyfills
5-6	Prototypes, classes, ES6+, modules	Small class-based library
7-8	DOM, events, fetch, storage	Weather app, notes app
9-10	Async deep dive, event loop, errors	Promise library, API client with retry

Weeks	Focus	Build
11	Node.js, npm, Express basics	REST API
12	TypeScript intro, testing (Jest/Vitest), a framework (React or Vue)	Full-stack project

Habits that create mastery: code daily, read other people's code, explain concepts aloud, solve one problem a day, build projects, read MDN and the ECMAScript spec, and debug with DevTools instead of guessing.

Best resources: MDN Web Docs, javascript.info, *You Don't Know JS* (Kyle Simpson), *Eloquent JavaScript* (Marijn Haverbeke), and Frontend Mentor or LeetCode for practice.

PART II: THE DEEP DIVE

Parts 1 to 10 gave you the full map. Part II goes underneath: how the engine really works, the exact rules behind the "weird" behaviour, async built from scratch, patterns, tooling, 45 more interview questions, and worked solutions.

---

11. How JavaScript actually runs

11.1 Execution contexts

Every time code runs, the engine creates an **execution context**. There are three kinds: global, function, and `eval`. Each context has two phases.

1. **Creation phase:** the engine builds the lexical environment, registers declarations (this is hoisting), binds `this`, and links to the outer environment.
2. **Execution phase:** code runs line by line, assignments happen.

A **lexical environment** is a record of variable names to values plus a pointer to its outer environment. The scope chain is simply this chain of pointers. A closure is a function object holding a pointer to the environment where it was created.

```
function outer() {
  const x = 'outer';
  function inner() { return x; }   // inner's [[Environment]] points to
outer's environment
  return inner;
}
outer()(); // 'outer'
```

11.2 The call stack

The stack is a last-in, first-out list of execution contexts. Calling a function pushes a frame; returning pops it. Infinite recursion fills it and throws `RangeError: Maximum call stack size exceeded`.

```
function a() { b(); }
function b() { c(); }
function c() { console.trace(); }
a(); // stack: c, b, a, global
```

11.3 Inside a modern engine (V8)

1. **Parse** source into an Abstract Syntax Tree (AST).
2. **Ignition** (interpreter) turns the AST into bytecode and starts running immediately.
3. Hot code is profiled; **TurboFan** (optimising JIT) compiles it into fast machine code using assumptions about types.
4. If an assumption breaks (a function that always got numbers suddenly gets a string), the engine **deoptimises** back to bytecode.

Practical consequences:

- Keep object shapes consistent. Objects created with the same properties in the same order share a **hidden class** (shape), enabling fast property access via **inline caches**.
- Do not add or delete properties randomly after creation; do not change a variable's type back and forth in hot paths.
- Arrays of one element kind (all small integers) are stored compactly; mixing types changes the representation.

11.4 Memory

- **Stack:** primitives and references (small, fast, automatic).

- **Heap:** objects, arrays, functions.
- **Generational garbage collection:** new objects live in the "young generation" and are collected often (scavenge); survivors move to the "old generation", collected less often with mark-sweep-compact, mostly incremental and concurrent to avoid pauses.

---

12. Type coercion and equality: the exact rules

12.1 The three conversions

ToString: `null` gives `'null'`, `undefined` gives `'undefined'`, `true` gives `'true'`, `[1,2]` gives `'1,2'`, `{}` gives `'[object Object]'`, `-0` gives `'0'`.

ToNumber:

Value	Number
`''` or whitespace only	`0`
`'42'`, `' 42 '`	`42`
`'42px'`	`NaN`
`true` / `false`	`1` / `0`
`null`	`0`
`undefined`	`NaN`
`[]`	`0` (via `''`)
`[5]`	`5`
`[1,2]`	`NaN`
`{}`	`NaN`

ToBoolean: only these are falsy: `false`, `0`, `-0`, `0n`, `''`, `null`, `undefined`, `NaN`. Everything else is truthy, including `'0'`, `'false'`, `[]`, `{}` and `function(){}`.

12.2 ToPrimitive

When an object meets an operator expecting a primitive, the engine calls `obj[Symbol.toPrimitive](hint)` if present; otherwise `valueOf()` then `toString()` (for hint `number` / `default`), or `toString()` then `valueOf()` (for hint `string`).

```
const money = {
  amount: 50,
  [Symbol.toPrimitive](hint) {
    return hint === 'string' ? `$${this.amount}` : this.amount;
  }
};
`${money}`;  // '$50'
money + 10;   // 60
```

12.3 The `+` operator

1. Convert both operands to primitives.
2. If either is a string, concatenate strings.
3. Otherwise convert both to numbers and add.

That is why `[] + {}` is `'[object Object]'` (both become strings) and `1 + null` is `1`, `1 + undefined` is `NaN`, `'3' * '4'` is `12`.

12.4 The `==` algorithm in plain words

- Same type: behaves like `===`.
- `null == undefined` is true, and they equal nothing else.
- Number vs string: string becomes number.
- Boolean vs anything: boolean becomes number first.
- Object vs primitive: object goes through ToPrimitive.
- `NaN` equals nothing, not even itself.

```
null == 0;        // false (null only equals undefined)
'0' == false;     // true  ('0' -> 0, false -> 0)
[] == ![];        // true  (![] is false -> 0; [] -> '' -> 0)
[2] == 2;         // true
NaN == NaN;       // false
```

Equality summary: `===` strict; `==` coercing; `Object.is` like `===` but `NaN` equals `NaN` and `+0` differs from `-0`; `Map` / `Set` use SameValueZero (`NaN` equals `NaN`, `+0` equals `-0`).

12.5 Floating point

Numbers are 64-bit IEEE 754 doubles. `0.1 + 0.2 = 0.30000000000000004`. Compare with a tolerance (`Math.abs(a - b) < Number.EPSILON`), work in integer cents for money, or use a decimal library. Integers are exact only up to `Number.MAX_SAFE_INTEGER` (9007199254740991). `BigInt` cannot be mixed with `Number` in arithmetic.

---

13. Objects and classes in depth

13.1 Property descriptors

Every property has attributes: `value`, `writable`, `enumerable`, `configurable` (or `get` / `set` for accessors).

```
const o = {};
Object.defineProperty(o, 'id', { value: 1, writable: false, enumerable:
false, configurable: false });
o.id = 2;            // silently ignored (TypeError in strict mode)
Object.keys(o);      // [] (not enumerable)
Object.getOwnPropertyDescriptor(o, 'id');
```

Method	Add props	Delete props	Change values
`Object.preventExtensions`	no	yes	yes
`Object.seal`	no	no	yes
`Object.freeze`	no	no	no

All three are shallow.

13.2 Getters, setters, computed keys

```
const temp = {
  _c: 25,
  get f() { return this._c * 9 / 5 + 32; },
  set f(v) { this._c = (v - 32) * 5 / 9; },
  ['key' + 1]: 'computed'
};
```

13.3 Creating objects: five ways

1. Literal `{}`
2. Constructor function with `new`
3. `Object.create(proto)`
4. `class`
5. Factory function (returns an object, uses closures for privacy, no `this` problems)

13.4 The prototype chain precisely

- `obj.__proto__` (legacy accessor) and `Object.getPrototypeOf(obj)` read an object's prototype.
- `Func.prototype` is the object that becomes the prototype of instances created with `new Func`.
- `instanceof` checks whether `Func.prototype` appears anywhere in the object's chain.
- Property **reads** walk the chain; property **writes** create an own property (shadowing) unless a setter or read-only property is found up the chain.
- Methods placed on the prototype are shared across all instances (memory efficient); methods defined in the constructor are per instance.

```
class A { hi() { return 'A'; } }
class B extends A { hi() { return super.hi() + 'B'; } }
new B().hi();                          // 'AB'
Object.getPrototypeOf(B) === A;        // true (static inheritance)
new B() instanceof A;                  // true
```

13.5 Classes: details worth knowing

- Class bodies are always strict mode.

- Methods are non-enumerable and are not bound; passing `obj.method` as a callback loses `this`. Fix with `bind`, an arrow wrapper, or an arrow class field (`handle = () => {}`).
- `super(...)` must be called before using `this` in a derived constructor.
- `#private` fields are truly private (enforced by the engine, unlike the `_underscore` convention).
- `static` members belong to the class; `static { }` blocks run once at definition.
- **Mixins:** `const Serializable = Base => class extends Base { toJSON() {} }`.
- **Composition over inheritance:** prefer assembling behaviour from small functions or objects rather than deep class trees.

13.6 Iterators and generators

```
const range = {
  from: 1, to: 3,
  [Symbol.iterator]() {
    let cur = this.from, end = this.to;
    return { next: () => cur <= end ? { value: cur++, done: false } : {
value: undefined, done: true } };
  }
};
[...range]; // [1, 2, 3]

function* idGen() { let id = 1; while (true) yield id++; }
const g = idGen(); g.next().value; // 1
```

Generators can receive values (`const x = yield y`), delegate (`yield*`), and are the foundation of lazy sequences and old-style `co`-like async flows.

---

14. Asynchronous JavaScript in depth

14.1 A promise from scratch (the interview favourite)

```
class MyPromise {
  #state = 'pending'; #value; #handlers = [];
  constructor(executor) {
    const resolve = v => this.#settle('fulfilled', v);
    const reject = e => this.#settle('rejected', e);
```

```
    try { executor(resolve, reject); } catch (e) { reject(e); }
  }
  #settle(state, value) {
    if (this.#state !== 'pending') return;       // settle once only
    this.#state = state; this.#value = value;
    queueMicrotask(() => this.#handlers.forEach(h => this.#run(h)));
  }
  #run({ onOk, onErr, resolve, reject }) {
    const cb = this.#state === 'fulfilled' ? onOk : onErr;
    if (!cb) return (this.#state === 'fulfilled' ? resolve : reject)
(this.#value);
    try { resolve(cb(this.#value)); } catch (e) { reject(e); }
  }
  then(onOk, onErr) {
    return new MyPromise((resolve, reject) => {
      const h = { onOk, onErr, resolve, reject };
      this.#state === 'pending' ? this.#handlers.push(h) : queueMicrotask(()
=> this.#run(h));
    });
  }
  catch(onErr) { return this.then(undefined, onErr); }
}
```

This simplified version omits thenable resolution (a returned promise being adopted), which the real specification requires. Know that gap if asked.

14.2 Key promise behaviours

- `.then` always returns a **new** promise; returning a value fulfils it, throwing rejects it, returning a promise adopts that promise.
- Handlers always run **asynchronously** (as microtasks), even if the promise is already settled.
- An unhandled rejection raises `unhandledrejection` (browser) or can crash Node (default since v15).
- `await x` is roughly `Promise.resolve(x).then(continuation)`.

14.3 Concurrency patterns

```
// Sequential
for (const id of ids) results.push(await fetchUser(id));

// Parallel, fail-fast
const all = await Promise.all(ids.map(fetchUser));

// Parallel, tolerate failures
const settled = await Promise.allSettled(ids.map(fetchUser));

// Limit concurrency to N
async function pool(tasks, limit) {
  const results = []; let i = 0;
  async function worker() {
    while (i < tasks.length) { const idx = i++; results[idx] = await
tasks[idx](); }
  }
  await Promise.all(Array.from({ length: limit }, worker));
  return results;
}

// Timeout with race
const timeout = (ms) => new Promise((_, rej) => setTimeout(() => rej(new
Error('timeout')), ms));
await Promise.race([fetch(url), timeout(5000)]);
```

14.4 Async iteration

```
async function* pages() { for (let p = 1; p <= 3; p++) yield await
fetchPage(p); }
for await (const page of pages()) console.log(page);
```

14.5 The Node.js event loop phases

Timers, pending callbacks, idle/prepare, poll (I/O), check (`setImmediate`), close callbacks. Between each callback, `process.nextTick` queue runs first, then promise microtasks. Order of priority: `nextTick` , then promises, then timers/immediates.

```
setTimeout(() => console.log('timeout'), 0);
setImmediate(() => console.log('immediate'));
process.nextTick(() => console.log('tick'));
Promise.resolve().then(() => console.log('promise'));
// tick, promise, then timeout/immediate (order of the last two is not
guaranteed in the main module)
```

14.6 Common async mistakes

- Using `forEach` with `async` callbacks and expecting it to wait.
- Forgetting `return` or `await` so errors escape the `try/catch`.
- Awaiting independent tasks sequentially (slow).
- Not checking `response.ok` after `fetch`.
- Race conditions: a slow older request overwriting a newer one; fix with `AbortController` or a request counter.

---

15. Patterns, tooling and the wider ecosystem

15.1 Design patterns every developer should recognise

- **Module:** closure or ES module exposing a public API, hiding internals.
- **Factory:** function that returns configured objects.
- **Singleton:** one instance (an ES module is already a singleton by default).
- **Observer / pub-sub:** subjects notify subscribers (`addEventListener`, EventEmitter, RxJS).
- **Strategy:** swap algorithms by passing functions.
- **Decorator / higher-order function:** wrap a function to add behaviour (logging, caching).
- **Iterator**, **Proxy**, **Dependency injection** (pass dependencies in, easy to test).
- **Immutability:** return new objects instead of mutating; foundation of Redux and React state.

15.2 Clean code habits

- Small pure functions, descriptive names, early returns, no magic numbers.
- Prefer `const`, strict equality, `async/await`, optional chaining.

- Handle errors deliberately; never swallow them silently.
- Avoid globals; avoid deep nesting; avoid mutating arguments.

15.3 Tooling map

Need	Tools
Runtime	Node.js, Deno, Bun
Packages	npm, pnpm, yarn
Bundling	Vite, esbuild, webpack, Rollup
Transpiling	Babel, SWC, TypeScript
Lint / format	ESLint, Prettier
Testing	Vitest, Jest, Playwright, Cypress
Frameworks	React, Vue, Svelte, Angular; Next.js, Nuxt, SvelteKit
Backend	Express, Fastify, NestJS
Types	TypeScript, JSDoc

15.4 Node.js essentials

```
import { readFile } from 'node:fs/promises';
import http from 'node:http';

const data = await readFile('./data.json', 'utf8');
http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify({ ok: true }));
}).listen(3000);
```

Know: `process.env`, `package.json` scripts, `node_modules` resolution, streams and buffers, `EventEmitter`, `child_process` and `worker_threads`, and why blocking the event loop with CPU-heavy synchronous code freezes the server.

15.5 Testing basics

```
import { test, expect } from 'vitest';
test('adds numbers', () => { expect(add(2, 3)).toBe(5); });
```

Unit tests for pure logic, integration tests for modules together, end-to-end tests for user flows. Mock network calls; test behaviour, not implementation.

15.6 TypeScript in five minutes

```
interface User { id: number; name: string; email?: string }
function greet(u: User): string { return `Hi ${u.name}`; }
type Result<T> = { ok: true; value: T } | { ok: false; error: string };
```

Key ideas: static types erased at compile time, interfaces and type aliases, generics, union and intersection types, narrowing with `typeof` / `in` /discriminants, `unknown` over `any`.

---

16. More interview questions (56 to 100)

16.1 Language depth

56. What are the execution phases of a function? Creation (hoisting, `this`, scope link) then execution.

57. Why is `typeof undeclaredVar` safe but `undeclaredVar` throws? `typeof` returns `'undefined'` for unresolvable references; direct access throws `ReferenceError`.

58. What is the difference between a function declaration and expression regarding hoisting? Declarations are hoisted with their body; expressions assigned to `var` hoist only the name as `undefined`, and to `let` / `const` stay in the TDZ.

59. What is a pure function vs a side effect? Give an example of each. Pure: `(a, b) => a + b`. Side effect: modifying a global, mutating an argument, logging, network I/O.

60. What are the differences between `slice` and `splice`? `slice(start, end)` returns a copy and leaves the original; `splice(start, count, ...items)` mutates and returns the removed items.

61. What does `Array.prototype.sort` do by default? Sorts by string (UTF-16) order, so `[10, 2, 1].sort()` gives `[1, 10, 2]`. Pass `(a, b) => a - b`.

62. Why does `parseInt('08')` matter? Historically parsed octal; always pass the radix: `parseInt('08', 10)`. Prefer `Number()` for strict conversion.

63. How do you check an empty object? `Object.keys(obj).length === 0` (and for arrays `arr.length === 0`).

64. How do you merge objects, and what happens with duplicate keys? `{ ...a, ...b }`; later keys win. Nested objects are not merged deeply.

65. What does `arguments` contain, and why avoid it? Array-like of all passed arguments in regular functions; not in arrows; prefer rest parameters.

66. What is a polyfill vs a transpiler? A polyfill adds a missing runtime API (e.g. `Array.prototype.at`); a transpiler rewrites newer syntax into older syntax (e.g. optional chaining).

67. What is the difference between `Object.keys`, `for...in` and `Object.getOwnPropertyNames`? `keys` returns own enumerable string keys; `for...in` includes inherited enumerable keys; `getOwnPropertyNames` includes non-enumerable own keys.

68. What is tail call optimisation, and does JavaScript have it? The spec includes it but only Safari implements it; do not rely on it. Convert deep recursion to loops.

69. What are tagged templates? A function called with the string pieces and interpolated values: `tag`a${1}b``. Used by styled-components and SQL helpers.

70. What is the difference between `Symbol()` and `Symbol.for()`? `Symbol()` is always unique; `Symbol.for(key)` shares one symbol per key in a global registry.

71. What is a `WeakRef` and `FinalizationRegistry`? A weak reference that does not stop garbage collection, and a callback after collection. Rarely appropriate; behaviour is nondeterministic.

72. What is the difference between `Object.assign` and spread? Nearly the same for own enumerable props; `assign` triggers setters on the target, spread defines new properties.

73. Explain `reduce` and implement `map` using it.

```
const map = (arr, fn) => arr.reduce((acc, x, i) => (acc.push(fn(x, i, arr)), acc), []);
```

74. What is lazy evaluation and how do generators help? Values are computed only on demand, enabling infinite sequences and memory-light pipelines.

75. What does `new.target` do? Inside a function it is the constructor called with `new`, or `undefined` if called plainly.

16.2 Browser and web

76. What happens when you type a URL and press Enter? DNS lookup, TCP/TLS handshake, HTTP request, response, HTML parsing into DOM, CSS into CSSOM, render tree, layout, paint, composite; scripts may block parsing unless `defer` / `async`.

77. What is the critical rendering path? The steps from bytes to pixels; optimise by minimising blocking CSS/JS and critical resources.

78. Reflow vs repaint? Reflow recalculates geometry (expensive); repaint redraws without geometry change. Batch DOM reads and writes to avoid layout thrashing.

79. What is the virtual DOM? An in-memory tree that a framework diffs against the previous version to apply minimal real DOM updates.

80. What is the same-origin policy? Scripts from one origin (scheme, host, port) cannot read data from another unless permitted (CORS, `postMessage`).

81. What are cookies attributes `HttpOnly`, `Secure`, `SameSite`? Block script access; send only over HTTPS; limit cross-site sending (CSRF defence).

82. What are the differences between `innerHTML`, `textContent`, `innerText`? `innerHTML` parses HTML (XSS risk); `textContent` raw text; `innerText` layout-aware and triggers reflow.

83. What is `IntersectionObserver` used for? Detecting element visibility: lazy loading, infinite scroll, analytics.

84. What is a Service Worker? A background script acting as a network proxy enabling offline use, caching, push notifications; basis of PWAs.

85. What is hydration (SSR)? Attaching event handlers and state to server-rendered HTML in the browser.

86. What are `Promise` microtasks vs `requestAnimationFrame`? Microtasks run right after the current script; rAF runs before the next paint.

87. How would you optimise a slow page? Measure first (Lighthouse, Performance tab); reduce bundle size (code splitting, tree shaking), compress, cache, lazy load, defer scripts, optimise images, avoid layout thrashing, use workers for CPU tasks.

16.3 Scenario and design

88. How would you implement an autocomplete search box? Debounce input, cancel stale requests with `AbortController`, cache results in a `Map`, handle loading/empty/error states, keyboard accessibility, escape output.

89. How would you handle a failing API with retry? Exponential backoff with jitter, cap attempts, retry only idempotent or safe calls and transient errors (network, 5xx, 429).

90. How do you structure a large JavaScript codebase? Feature-based folders, ES modules with clear public APIs, shared utilities, consistent lint/format/test pipeline, TypeScript, code review, CI.

91. How do you share state between components? Props and callbacks, context, a store (Redux, Zustand, Pinia), URL state, server state libraries (TanStack Query).

92. How would you detect and fix a memory leak? Heap snapshots before and after an action in DevTools; look for growing detached DOM nodes and retained closures; remove listeners, clear timers, abort requests on teardown.

93. How do you secure a Node API? Validate input, parameterised queries, rate limiting, helmet headers, CORS allow-list, hashed passwords (bcrypt/argon2), short-lived tokens, dependency audits, secrets in environment variables.

94. Write a function to run promises in series.

```
const series = tasks => tasks.reduce((p, t) => p.then(acc => t().then(r =>
[...acc, r])), Promise.resolve([]));
```

95. What would you do if `await` is used inside `forEach`? Replace with `for...of` (sequential) or `await Promise.all(arr.map(async x => ...))` (parallel).

96. How do you make a function run only once?

```
const once = fn => { let done = false, r; return (...a) => done ? r : (done =
true, r = fn(...a)); };
```

97. How do you make an object deeply immutable? Recursively `Object.freeze` its properties, or use persistent data libraries (Immer, Immutable.js).

98. Compare callbacks, promises, observables. Callbacks: one-shot, inversion of control. Promises: one future value, chainable, eager. Observables: zero or many values over time, lazy, cancellable (RxJS).

99. When would you pick Node.js, and when not? Good for I/O-bound, real-time, API and tooling work; poor for CPU-bound number crunching unless using worker threads or other services.

100. How do you stay current with JavaScript? TC39 proposals on GitHub, MDN, release notes for runtimes and frameworks, State of JS survey, building small projects with new features.

---

17. More output puzzles with explanations

```
// P11
console.log([10, 9, 1].sort());           // [1, 10, 9]  (string sort)

// P12
const a = { n: 1 }; const b = { n: 1 };
console.log(a == b, a === b);            // false false  (different
references)

// P13
function f() { return this; }
console.log(f() === globalThis);          // true in sloppy mode, false
(undefined) in strict

// P14
const obj = { name: 'x', arrow: () => this.name, reg() { return this.name; }
};
console.log(obj.reg());                   // 'x'; obj.arrow() is not 'x'
(arrow takes outer this)

// P15
console.log(1 + +'2' + '3');              // '33'

// P16
console.log(typeof typeof 1);             // 'string'

// P17
let i = 0; const r = [i++, i++, ++i];
```

```
console.log(r);                              // [0, 1, 3]

// P18
console.log([..'abc'].map((c, i) => c + i)); // ['a0', 'b1', 'c2']

// P19
console.log(Boolean('false'), Boolean(''), Boolean([]));   // true false true

// P20
Promise.reject('x').catch(e => { console.log(e); return 'y';
}).then(console.log); // x then y
```

---

18. Worked solutions to the coding challenges

```
// 1. Reverse and palindrome
const reverse = s => [...s].reverse().join('');
const isPalindrome = s => { const c = s.toLowerCase().replace(/[^a-z0-9]/g,
''); return c === reverse(c); };

// 2. Duplicates
const unique = arr => [...new Set(arr)];
const duplicates = arr => { const seen = new Set(), dup = new Set(); for
(const x of arr) seen.has(x) ? dup.add(x) : seen.add(x); return [...dup]; };

// 3. Flatten (recursive, depth-aware)
const flatten = (arr, depth = Infinity) => arr.reduce((acc, x) =>
  Array.isArray(x) && depth > 0 ? acc.concat(flatten(x, depth - 1)) :
acc.concat(x), []);

// 4. Throttle, memoize, curry
function throttle(fn, ms) {
  let last = 0;
  return function (...a) { const now = Date.now(); if (now - last >= ms) {
last = now; fn.apply(this, a); } };
}
function memoize(fn) {
  const cache = new Map();
  return (...a) => { const k = JSON.stringify(a); if (!cache.has(k))
cache.set(k, fn(...a)); return cache.get(k); };
}
function curry(fn) {
```

```
  return function c(...a) { return a.length >= fn.length ? fn.apply(this, a)
: (...b) => c.apply(this, [...a, ...b]); };
}

// 5. Polyfills
Array.prototype.myFilter = function (cb, thisArg) {
  const out = [];
  for (let i = 0; i < this.length; i++) if (i in this && cb.call(thisArg,
this[i], i, this)) out.push(this[i]);
  return out;
};
Array.prototype.myReduce = function (cb, init) {
  let i = 0, acc = init;
  if (arguments.length < 2) { if (!this.length) throw new TypeError('Empty
array'); acc = this[i++]; }
  for (; i < this.length; i++) acc = cb(acc, this[i], i, this);
  return acc;
};

// 6. bind polyfill
Function.prototype.myBind = function (ctx, ...pre) {
  const fn = this;
  return function (...args) { return fn.apply(ctx, [...pre, ...args]); };
};

// 7. Deep clone and deep equal
function deepEqual(a, b) {
  if (Object.is(a, b)) return true;
  if (typeof a !== 'object' || typeof b !== 'object' || !a || !b) return
false;
  const ka = Object.keys(a), kb = Object.keys(b);
  return ka.length === kb.length && ka.every(k => deepEqual(a[k], b[k]));
}

// 8. Promise.all
function myAll(promises) {
  return new Promise((resolve, reject) => {
    const results = []; let left = promises.length;
    if (!left) return resolve(results);
    promises.forEach((p, i) => Promise.resolve(p).then(v => { results[i] = v;
if (--left === 0) resolve(results); }, reject));
  });
}
```

```
// 9. Event emitter
class Emitter {
  #h = new Map();
  on(e, fn) { (this.#h.get(e) ?? this.#h.set(e, new Set()).get(e)).add(fn);
return () => this.off(e, fn); }
  off(e, fn) { this.#h.get(e)?.delete(fn); }
  emit(e, ...a) { this.#h.get(e)?.forEach(fn => fn(...a)); }
}

// 10. Two-sum, anagram, first unique char
function twoSum(nums, target) {
  const seen = new Map();
  for (let i = 0; i < nums.length; i++) {
    if (seen.has(target - nums[i])) return [seen.get(target - nums[i]), i];
    seen.set(nums[i], i);
  }
}
const isAnagram = (a, b) => [...a].sort().join('') ===
[...b].sort().join('');
const firstUnique = s => [...s].find(c => s.indexOf(c) === s.lastIndexOf(c));

// 11. sleep and retry with backoff
const sleep = ms => new Promise(r => setTimeout(r, ms));
async function retry(fn, tries = 3, delay = 300) {
  for (let i = 0; ; i++) {
    try { return await fn(); }
    catch (e) { if (i >= tries - 1) throw e; await sleep(delay * 2 ** i); }
  }
}

// 12. LRU cache (Map keeps insertion order)
class LRU {
  constructor(cap) { this.cap = cap; this.m = new Map(); }
  get(k) { if (!this.m.has(k)) return undefined; const v = this.m.get(k);
this.m.delete(k); this.m.set(k, v); return v; }
  put(k, v) { this.m.delete(k); this.m.set(k, v); if (this.m.size > this.cap)
this.m.delete(this.m.keys().next().value); }
}
```

19. Mastery checklist

Tick each only when you can explain it aloud without notes and write an example from memory.

- [ ] Scope, hoisting, TDZ, closures, and the `var` loop trap
- [ ] All five `this` rules, plus `call`, `apply`, `bind`
- [ ] Coercion rules, `==` algorithm, falsy list
- [ ] Prototype chain, `new`, classes, `instanceof`, private fields
- [ ] Immutability, shallow vs deep copy, `freeze` / `seal`
- [ ] Event loop with microtasks and macrotasks; predict output order
- [ ] Promises, `async/await`, `Promise.all` family, error handling, cancellation
- [ ] Iterators, generators, async iteration
- [ ] DOM, events, delegation, storage, `fetch`
- [ ] Modules (ESM vs CommonJS), bundling, tree shaking
- [ ] Security: XSS, CSRF, CORS, supply chain
- [ ] Performance: reflow, debounce/throttle, memory leaks, profiling
- [ ] Node.js basics, testing, TypeScript basics
- [ ] Implement from memory: debounce, throttle, memoize, curry, bind, `Promise.all`, event emitter, LRU cache, deep clone